

Haneen Alaa Hassan

Day 2: c#

(part 1)

Q1.

What is the shortcut to comment and uncomment a selected block of code in Visual Studio?

Answer:

To comment selected code: Ctrl + K, Ctrl + C

To uncomment selected code: Ctrl + K, Ctrl + U

Q2.

Explain the difference between a runtime error and a logical error with examples.

Answer:

A runtime error occurs while the program is running and usually causes an exception.

Example:

```
int x = 10;
```

```
int y = 0;
```

```
Console.WriteLine(x / y);
```

This causes a DivideByZeroException.

A logical error occurs when the program runs successfully, but produces an incorrect result because the logic is wrong.

Example:

```
int x = 10;
```

```
int y = 20;
```

```
int sum = x - y;
```

The program runs, but the result is incorrect because + should be used instead of .

Q3.

Why is it important to follow naming conventions such as PascalCase in C#?

Answer:

Naming conventions make code readable, consistent, and easier to maintain. They also help developers understand the purpose of classes, methods, and variables quickly.

For example:

```
class Student
```

```
{  
void CalculateSalary()  
{  
}  
}
```

PascalCase is commonly used for classes and methods, while camelCase is commonly used for local variables and parameters.

Q4.

Explain the difference between value types and reference types in terms of memory allocation.

Answer:

Value types store their actual value. When a value type is assigned to another variable, the value is copied.

Example:

```
int x = 10;
```

```
int y = x;
```

```
y = 20;
```

Here, x remains 10.

Reference types store a reference to an object. When one reference is assigned to another, both references can point to the same object.

Example:

```
Person p1 = new Person();
```

```
Person p2 = p1;
```

Both p1 and p2 refer to the same object.

It is an oversimplification to say that value types are always on the stack and reference types are always on the heap. The important difference is how their values/references are stored and copied.

Q5.

What will be the output of the following code? Explain why.

```
int a = 2, b = 7;
```

```
Console.WriteLine(a % b);
```

Answer:

The output is:

2

The % operator returns the remainder after division.

2 ÷ 7 gives 0 with a remainder of 2.

Therefore:

$$2 \% 7 = 2$$

Q6.

How does the && (logical AND) operator differ from the & (bitwise AND) operator?

Answer:

&& is the logical AND operator. It is mainly used with Boolean expressions and uses short-circuit evaluation.

Example:

```
if (age > 18 && isStudent)
{
}
```

Both conditions must be true.

& is the bitwise AND operator when used with integers. It compares the corresponding bits of two numbers.

Example:

$$5 = 0101$$

$$3 = 0011$$

$$0001 = 1$$

So:

5 & 3

returns 1.

Q7.

Why is explicit casting required when converting a double to an int?

Answer:

Explicit casting is required because converting a double to an int may cause data loss, such as losing the decimal part.

Example:

```
double x = 10.75;
```

```
int y = (int)x;
```

The result is:

10

The .75 is discarded.

Therefore, C# requires us to explicitly tell the compiler that we understand the possible data loss:

```
(int)x
```

Q8.

What exception might occur if the input is invalid when using Parse, and how can you handle it?

Answer:

When using `int.Parse()`, if the input isn't a valid integer, a `FormatException` can occur.

Example:

```
int age = int.Parse("abc");
```

This causes:

`FormatException`

We can handle it using try-catch:

```
try
{
    int age = int.Parse(input);
}
catch (FormatException)
{
    Console.WriteLine("Invalid input.");
}
```

`Parse` can also throw an `OverflowException` if the number is outside the range of `int`.

Q9.

Given the code below, what is the value of x after execution? Explain why.

```
int x = 5;
```

```
int y = ++x + x++;
```

Answer:

The final value of x is:

7

Step by step:

1. ++x increments x first:

$x = 6$

value used = 6

2. x++ uses the current value first:

value used = 6

then x becomes 7

Therefore:

$y = 6 + 6$

$y = 12$

$x = 7$

Final answer:

$x = 7$

$y = 12$

Compiled vs Interpreted Languages

1. Difference between Compiled and Interpreted Languages

Compiled Language:

The source code is translated by a compiler before execution.

Example: C, C++.

Interpreted Language:

The code is executed at runtime by an interpreter.

Example: JavaScript, Python.

C#

C# is a compiled language. The C# compiler converts C# code into IL (Intermediate Language), then the JIT (Just-In-Time) compiler converts IL into machine code at runtime.

C# Flow:

C# Code → Compiler → IL → JIT → Machine Code

2. Implicit vs Explicit vs Convert vs Parse

Implicit:

Automatic conversion by C#.

```
int x = 10;
```

```
double y = x;
```

Explicit:

Manual conversion using (type).

```
double x = 10.5;
```

```
int y = (int)x;
```

Result: 10

Convert:

Uses the Convert class to convert between types.

```
int x = Convert.ToInt32("10");
```

Parse:

Converts a string into a specific type.

```
int x = int.Parse("10");
```

If the string is invalid, Parse can throw a FormatException.

Summary

Implicit → Automatic

Explicit → (type)

Convert → **Convert.ToInt32()**

Parse → **int.Parse()**

